

Capítulo 4

1._F

No compila porque se está queriendo guardar un int dentro de un String, lo cual no es posible a menos que se convierta la sumatoria del int a String entonces podría compilar.

2._C, E y F

En la C se está usando una referencia como un tipo de dato, lo cual no es correcto, se está confundiendo un tipo de dato con el nombre de la variable.

En la E no se le está dando un tamaño al arreglo, además de que solo está considerando un [] cuando debería considerar ambos porque así lo establece el tipo de dato.

Y finalmente en la F, al igual que en la E no se está considerando ningún tamaño al arreglo, al menos la primera casilla [] debería estar lleno, porque se vuelve obligatoria asignarla ya que Java al menos pide que esté definida las filas, aunque con la segunda no hay tanto problema si se deja vacía, puedes más adelante definir las columnas.

3._A, C y D

La A se vuelve correcta debido que aunque no exista la hora 2:15, el ZonedDateTime lo ajusta a una hora válida, en este caso lo movería a 3:15.

En la C está bien, porque la hora 2:15 si llega a existir aunque repetido, debido a que en el momento que se atrasa el reloj vuelve a dar las 2:00 y por lo tanto pasa por segunda vez a dar 2:15.

En la D, es un día normal en donde el trayecto del tiempo sigue sin ningún cambio, por ende si pasa por la hora 2:15pm

En las demás hay fechas incorrectas, y además el MonthEnum es incorrecto porque no existe.

4._A, C y D

A esta bien pq el if está comparando lo que se encuentra dentro del objeto, lo cual es lo mismo "Hello". En la C con el intern() ahora tanto t como s apuntan al mismo objeto. En la D tanto "Hello" como s son el mismo objeto.

Las demás son incorrectas porque no están apuntando al mismo objeto, mantiene objetos distintos.

5._B

En este comienza imprimiendo aaa y cada una se coloca en su posición correspondiente [0][1][2], pero después la siguiente instrucción dice que en el índice[1] inserte bb, por lo tanto queda como abbaa y se reacomoda sus posiciones a[0]b[1]b[2]a[3]a[4], luego viene la inserción en la posición [4] de ccc, lo que finalmente nos deja con el resultado de abbacca.

6._C

Hay 2 líneas con error debido a que, la primera de 'int two = Math.round(1.0);' se recibe un double y se regresa un long lo cual es imposible guardar en un int, porque el long rebasa la longitud del int. y en la siguiente línea 'float three = Math.random();' sucede algo similar, se regresa un double pero no lo puedes meter en un float sin haber hecho un casting primero, cosa que no sucede y por ende arroja un error de compilación.

7. _A y E

Aquí primero tenemos que convertir las horas de acuerdo a lo establecido al GMT, pero aquí en lugar de restar las horas, se suman, entonces: T05:00 GMT-04:00 queda en la hora de 9:00 y el: T09:00 GMT- 06:00 queda en 15:00, entonces la primera date/time es más temprano (A) y hay 6hrs de diferencias entre las dos (E).

8. _A, B y F

En la opción A si retorna 5 ya que está en la posición [4], en la B primero reemplazamos el 3 y 4 por el 6, esto debido a que `replace(2, 4, "6")` nos está indicando que reemplazamos desde el índice [2] hasta antes del [4], o sea en el índice [3], entonces al final tenemos 1265 y por ende si imprime 5 por el `.charAt(3)`. Finalmente en el F tenemos que `replace("123", "1")` nos deja con el resultado de 145 y con `charAt(2)` retorna 5.

En las demás da error de compilación o devuelven otros números.

9. _A, C y F

Los array comienzan desde el índice [0] y no como normalmente nosotros iniciamos la enumeración que es del 1 en adelante, por ello mismo la B está mal.

En la C es cierto que una vez establecido el tamaño del arreglo no se puede cambiar y además son mutables, por ello la D es incorrecta.

La F indica que el `.equals()` siempre da false, esto porque lo que comparan es la referencia y no su contenido, por ello la E y G es incorrecta.

10. _A

Ninguna línea falla en compilación, en las primeras tres líneas todo compila correctamente ya que devuelven el mismo tipo de dato en que se guarda, y en la última línea, Java hace la conversión de int, long y double a double y así todas caben sin problemas

11. _E

No compila debido a un error en el código, y es que la línea `date.plusHours(3);` nos indica agregarle 3 horas a la fecha, pero `LocalDate` no maneja horas, solo año, mes y día. Luego la línea `date.plusDays(2);` nos indica aumentar 2 días más a la fecha, sin embargo `LocalDate` es inmutable, por ende esa línea crea un objeto nuevo con esos días extras pero no se guarda en ningún lado, por ende sigue imprimiendo la fecha 3 de abril.

12. _A, D y E

A es correcto debido a que la línea `numbers.substring(1, 3)` nos indica que tomemos los índices [1] y [2], esto debido a que el índice fin se excluye, por ende el índice [3] no se toma en cuenta, entonces imprime 12. Luego en la segunda línea (`numbers.substring(7, 7)`) imprime una línea en blanco ya el espacio entre el 7 y 7 no existe, no hay nada. finalmente en la línea `numbers.substring(7)` nos indica que tomemos desde el índice [7] hasta terminar el arreglo, por lo que solo queda el índice [8] y el resultado final es 78.

13. _B

Se obtiene al final `roar roar!!!` Debido a que el String es inmutable por lo tanto en ningún momento se modifica, a diferencia del `StringBuilder` se modificó y pasó de ser `roar` a `roar!!!`.

14. _A y F

Con el instant debemos buscar un punto exacto en el tiempo, por ende necesitamos algo que nos de fecha, hora, y la zona horaria, por ello A y F cumplen con todo eso, mientras que las demás opciones solo nos dan únicamente la fecha, o les falta la zona, lo cual se vuelve incorrecto.

15. _C y E

Java ordena los String de acuerdo al formato ascii, en donde nos dice a grandes rasgos que primero van los números, luego las mayúsculas y al final las minúsculas, por ende los valores ordenados para el primer print queda como: [123,PIG,pig], y ahora para el segundo print ya tenemos el array ordenado, sin embargo el .binarySearch lo que hace es buscar donde debería de estar en este caso "Pippa" y retorna el índice si lo encuentra, en caso contrario calcula donde debería de ir y nos retorna ese índice, que para este caso seria -3 debido a que tendría que ir en el índice 2 y siguiendo las reglas de Java nos dice que resultado-(2)-1 lo que es igual a -3.

16. _A, B y G

En la primera línea obtenemos el resultado de 11, esto debido a que contiene lo siguiente:

ewe = 3 caracteres

\n = 1 carácter (salto de línea)

sheep = 5 caracteres

\\ = 1 carácter (una diagonal invertida \)

t = 1 carácter

y así obtenemos un total de: 11 caracteres

Luego, en la segunda línea usamos indent(2), el cual agrega 2 espacios al inicio de cada línea. Como hay 2 líneas, se agregan 4 caracteres. Además, el método agrega un salto de línea al final si no existe, sumando 1 carácter más. Entonces el segundo resultado nos da:

11 + 5 = 16

Finalmente, translateEscapes() convierte texto escapado en caracteres reales. Por ejemplo, "\\t" (2 caracteres) se convierte en "\t" (1 carácter). Por lo tanto, se reduce un carácter y nos queda 11 - 1 = 10

17. _A y G

La A es correcta debido a que substring(1, 2), nos indica que comienza en índice [1] que es igual a "b" y termina antes del índice [2] que sigue siendo solo el índice [1], por ende el resultado es: "b" (1 carácter)

En la G si lanza una excepción debido a que inicio es mayor que el final.

Y finalmente, en la B regresa un String vacío, en la C es inválido, en la D no hay un carácter, en la E si retorna un carácter válido, en la F no hay errores solo nos retorna un vacío, y la H nuevamente no lanza un error

18. _C y F

s1 empieza como "purr" que es igual a 4 caracteres, después tenemos a los métodos toUpperCase(), trim() y substring() pero estos no cambian nada porque String es inmutable. Luego con s1 += "two" pasa a ser "purrtwo" lo cual nos da un resultado de 7 caracteres, por eso C es correcta.

Luego pasamos a s2 que este se va formando de la siguiente manera: comenzamos con "2", luego concatenamos 'c' que nos da "2c" y finalmente concatenamos a ello 'false' y nos

queda como: "2cfalse". Después en la línea `s2 == "2cfalse"` compara referencias, lo cual nos da false, pero en la línea `s2.equals("2cfalse")` compara contenido, lo cual es correcto e imprime "equals".

19. _A, B y D

En `compare()` nos dice si el primer arreglo es mayor que el segundo.

En este caso en A sí pasa, porque en la posición 1 "Peacock" va después que "Llama", entonces `s1` es mayor y da un número positivo.

En C no, porque `s4` es más largo. Y en E tampoco, porque son iguales y eso da 0.

Luego en `mismatch()` nos dice en qué posición son diferentes.

En B sí, porque en el índice [1] son diferentes y regresa 1.

En D también, porque en el índice [1] uno tiene "Llama" y el otro null, entonces también regresa 1.

En F no, porque no hay diferencias y regresa -1.

20. _A y D

Primero `dateTime1` empieza en 1:30 del 13 de marzo en zona US/Eastern.

Después hacemos `plus(1 hora)`, entonces en teoría debería ser 2:30, pero aquí pasa algo importante: ese día es cuando cambia el horario de verano, entonces las 2:00 a 2:59 no existen, el reloj se brinca directo a las 3:00, por eso realmente termina en 3:30. Luego: `diff` sí vale 1, porque aunque el reloj brincó, solo avanzamos 1 hora real, por ende A es correcto.

D es correcto porque `hour` es 3, porque el nuevo tiempo es 3:30

La E es incorrecto debido a que `offset` cambia porque entró el horario de verano, entonces ya no es el mismo.

21. _A y C

Primero tenemos `puzzle = "Java"`. Con la opción A que es `reverse()` simplemente invierte el texto a "avaJ", lo cual es correcto.

Ahora con el segundo caso es un poco más largo, que le pertenece a la opción C y en este primero hace `append("vaJ$")` que nos da "JavavaJ\$", luego `delete(0, 3)` borra "Jav" y nos da el resultado: "avaJ\$", después `deleteCharAt(puzzle.length() - 1)` borra el último carácter \$ y finalmente nos queda "avaJ".

Las demás. Las demás, como B es errónea porque `substring()` regresa un String pero no lo guarda, entonces `puzzle` no cambia, en la D se usa `deleteCharAt(puzzle.length())` y eso da error porque ese índice no existe.

22. _A

Aquí primero `date` empieza como 30 de abril de 2022.

Luego se llaman `plusDays(2)` y `plusYears(3)`, pero aquí lo importante es que `LocalDate` es inmutable, entonces esos métodos no cambian el objeto original, solo regresan uno nuevo y aquí no se guarda. O sea, `date` se queda igual todo el tiempo.

Entonces al imprimir sigue siendo la misma fecha desde el inicio, o sea: año = 2022, mes = APRIL y día = 30

Capítulo 5

1._ A y E

En final significa que ya no puedes cambiar el valor después de asignarlo. Entonces:

A) sí es correcta debido a que las variables tanto de instancia como estáticas sí pueden ser final y E) También es correcta porque si un primitivo es final, ya no puedes cambiar su valor.

Las demás son incorrectas debido a que por ejemplo en la B) “effectively final” no significa que tenga la palabra final, sino que no cambia su valor aunque no esté declarado así, en la C) es también incorrecta porque si un objeto es final, lo que no cambia es la referencia, pero sus datos sí se pueden modificar, y finalmente la D) sí puedes usar final junto con var

2._ B y C

Aquí el espacio en blanco va antes de void, entonces solo pueden ir modificadores, no cualquier otra cosa, por ende la opción B) final sí se puede poner antes de void, entonces compila y la opción C) private, también es un modificador válido para el método.

Los demás:

A) default, no se usa así para métodos, el acceso por default es no poner nada, en la D) Public Java no acepta el Public con mayusculas, debería ser public, en la E) String sería tipo de retorno, pero ya tenemos void y finalmente F) zzz este es como una etiqueta, no va en métodos.

3._ A y D

Aquí lo importante a notar es el orden, entonces primero van modificadores (public, private, static, final), luego el tipo de retorno, y luego el nombre, por ende la opción A) final static void rain() cumplen con este porque los modificadores pueden ir en cualquier orden y compila, y la D) static final void sleet() es igual, modificadores en distinto orden pero válido, esto también compila.

En los demás, por ejemplo en la B) hay int y void, dos tipos de retorno, lo cual no es correcto, en la C) es igual, void int, dos tipos, entonces está mal, luego la E) void final, esta mal el orden, el tipo va después de modificadores, y finalmente la F) void public igual esta mal el orden, public debería ir antes.

4._ A, B, C y E

Aquí tenemos final ____ song = 6, entonces el tipo tiene que aceptar el valor 6.

El 6 es un entero, entonces puede quedarse como int, o también puede convertirse a otros tipos más grandes como long o double, por eso A, C y E sí funcionan, también puede convertirse a Integer automáticamente (autoboxing), por eso B también funciona.

Pero en Long y Double no funciona, porque ahí Java tendría que primero convertir el 6 a otro tipo y luego hacer autoboxing, y eso necesitaría más código.

5._ A, C y D

Básicamente, si el método es void, sí puede tener return; pero sin regresar nada, o incluso no tener return y no pasa nada. Por eso A y C están bien.

Luego, si el método es int, ahí sí a fuerza tiene que regresar un número entero. Por eso D está bien porque regresa 9.

Los otros no funcionan porque, B intenta regresar null en un int, y eso no se puede porque null es para objetos, luego E regresa 9.0 que es double, no int. Y F no regresa nada aunque dice que es int, entonces le falta valor.

6._A, B y F

Aquí lo importante es lo de los ... (varargs): solo puede haber uno y tiene que ir hasta el final sí o sí.

Entonces A está bien porque cumple eso tal cual. B también porque primero pone un parámetro normal y luego el varargs al final, todo correcto. Y F también pasa porque ni usa varargs, son arreglos normales, entonces no hay bronca.

Los demás no jalan porque: C pone el varargs y luego otro parámetro después, y eso no se puede. D mete dos varargs y tampoco se puede. Y E está mal escrito porque el ... va después del tipo, no antes.

7._D y F

Aquí el método regresa b2.length, entonces lo único que importa es cuántos valores le mandas después del primer boolean.

En D sí salen 2 porque mandas true, true, true, entonces el primero es b y los otros dos se van al varargs de tamaño 2.

En F también porque le pasas new boolean[2], o sea ya le estás mandando un arreglo de tamaño 2 directamente y también da 2.

Los demás no dan 2 porque:

A ni compila, le falta el primer parámetro.

B manda solo uno, entonces el varargs queda vacío.

C manda uno extra de tamaño 1.

y E no compila porque el arreglo está mal escrito.

8._D

La respuesta es D.

O sea, sí puedes usar modificadores de acceso para dejar todos los atributos privados y todos los métodos públicos, y eso es súper común de hecho.

Las demás no:

A está mal porque protected es más permisivo que package, no al revés.

B está mal porque la clase es public, entonces sí la pueden ver desde fuera, aunque no puedan usar sus cosas internas.

C no se puede porque el acceso por package aplica a todo el paquete, no a ciertas clases.

E tampoco porque Java no tiene algo así como reglas por nombre tipo "los que empiezan con Test".

9._B, C, D y F

La respuesta es B, C, D y F.

O sea, el problema aquí es que las clases están en paquetes diferentes, entonces ya no puedes acceder a todo como si nada. La línea 5 falla porque globalKey tiene acceso de package (no es public), entonces desde otro paquete no se puede ver.

La línea 6 también falla porque el constructor tampoco es public, entonces no puedes crear el objeto desde fuera del paquete. La línea 7 falla porque roomNumber es private, entonces ni siquiera dentro del mismo paquete podrías acceder, menos aquí.

Y la línea 9 falla porque teacherName es protected, pero como no hay herencia entre clases, tampoco se puede usar desde otro paquete.

La única que sí pasa sin problema es la 8 porque floor sí es public.

10._B

La respuesta es B.

O sea, primero cuando se carga la clase Rope, se ejecuta todo lo estático. Entonces empieza con LENGTH = 5, pero luego entra al bloque static y lo cambia a 10, así que ese es el valor final. Luego en main:

Rope.swing(); imprime "swing "

new Rope().swing(); también imprime "swing " (aunque no es lo correcto usarlo así, sí funciona)

Y al final imprime LENGTH, que ya quedó en 10.

11._B y E

Hay un error de compilación porque en el método play() (que es static) se intenta llamar a climb(), que es de instancia, y eso no se puede. Por eso solo hay un error. Si quitas esa línea, el programa sí corre y se imprimen dos veces "swing", porque play() es static (aunque el objeto sea null no pasa nada). Entonces la salida es swing- swing

12._B

La respuesta es B.

O sea, una variable "effectively final" es básicamente una que solo se asigna una vez y ya no se vuelve a cambiar.

Entonces aquí:

monkey no cuenta porque lo modifican con el monkey++.

giraffe sí cuenta porque se asigna una vez y ya no se toca.

el name de adentro del if también cuenta porque igual solo se asigna una vez.

pero el otro name no cuenta porque luego le cambian el valor a null.

y food tampoco porque lo modifican dentro del while.

13._D

La respuesta es D.

O sea, aquí hay dos cosas clave:

Primero, ese bloque con {} es un initializer de instancia, no estático. Entonces nunca se ejecuta porque en ningún momento hacen new RopeSwing().

Segundo, length es static, eso significa que todos los objetos comparten el mismo valor. No importa si usas rope1 o rope2, es la misma variable.

Entonces en el main:

primero le ponen 2, pero luego le ponen 8, y como es la misma variable, se queda en 8.

Y eso es lo que imprime al final.

14._E

La respuesta es E (4 errores).

O sea, aquí todo gira en que los static final solo se pueden asignar una vez, ya sea cuando los declaras o dentro de un bloque static, pero ya después de eso no los puedes tocar otra vez.

Entonces: primero, bench está mal desde el inicio porque nunca lo inicializan donde deberían (ni en la declaración ni en static).

Luego, name ya tenía valor desde el principio y en el segundo static lo vuelven a asignar... eso ya no se puede.

también rightRope lo asignan en un static y luego otra vez en otro static, o sea doble asignación → error. y al final en el main intentan darle valor a bench, pero como es final, ya es demasiado tarde para hacerlo ahí.
entonces básicamente el error aquí es querer asignar más de una vez o fuera de lugar a variables final.

15._B

La respuesta es B.

O sea, aquí el punto es que quieres usar sort(list) así directo, sin poner Collections.sort(...), entonces necesitas un static import bien hecho.

La forma correcta es:

```
import static java.util.Collections.*;
```

porque eso te deja usar todos los métodos estáticos de Collections sin escribir el nombre de la clase.

Ahora, los demás:

A esta mal, porque estás intentando hacer static import de la clase completa, y eso no se puede.

C nada que ver, no puedes poner parámetros en un import.

D, E, F están mal porque escriben static import en vez de import static.

entonces básicamente: si quieres usar sort() directo, necesitas traerlo con import static, y la opción correcta aquí es B.

16._E

La respuesta es E (int- Object- Object-).

O sea, aquí todo es por cómo Java decide qué método usar (overloading + promoción/autoboxing).

Entonces:

t.print(s); s es short, y como no hay método con short, Java lo sube a int e imprime "int"

t.print(true); es boolean, y no hay método con boolean, entonces lo convierte a Boolean (autoboxing), cae en Object e imprime "Object"

t.print(6.789); eso es double, tampoco hay método para double, entonces lo convierte a Double y otra vez cae en Object e imprime "Object"

17._B

La respuesta es B (9).

O sea, aquí la clave es entender que en Java todo se pasa por valor, no por referencia.

Entonces:

value empieza siendo 9

se lo mandas a square(value), pero ahí adentro lo que recibe es una copia (x = 9)

dentro del método: calcula y = 81

luego cambia x a -1, pero eso solo afecta a la copia, no al value original y aunque result sí guarda 81, nunca lo imprimen

18._B, D y E

La respuesta es B, D y E. O sea, aquí lo que pasa es que en Java aunque trabajes con objetos, igual se pasa por valor, entonces cuando mandas s1 como parámetro, en el método hacen `a = new StringBuilder("a")`, pero eso solo cambia la copia, no el original, por eso s1 se queda igual como "s1". En cambio con s2, cuando hacen `b.append("b")`, ahí sí están modificando el objeto original porque siguen apuntando al mismo, entonces s2 termina siendo "s2b". Y al final, el método regresa ese nuevo "a" que crearon, entonces s3 queda como "a".

entonces básicamente queda: `s1 = s1`, `s2 = s2b`, `s3 = a`

19. _B, C y E

La respuesta es B, C y E. O sea, aquí todo depende de qué tipo de variable es cada una y desde dónde la estás intentando cambiar. `value1` es final, entonces solo se puede asignar una vez, y ya se hizo desde el inicio con "red", así que intentar cambiarla después (como en A o D) ya no se puede. Luego `value2` es static, y esa sí la puedes modificar tanto en el bloque normal como en el static, por eso B y E sí jalan. Y `value3` es de instancia, entonces sí la puedes cambiar en el bloque normal (C), pero no en el static, porque ahí no tienes acceso a variables de instancia, por eso F no compila.

entonces básicamente: static = flexible en ambos bloques, instancia = solo en bloque normal, final = una sola vez y ya

20. _A y E

La respuesta es A y E. O sea, aquí todo es por cómo Java decide qué método usar dependiendo del tipo y las prioridades. Primero con `Run.execute(100)`, como 100 es un int, se va directo al método que recibe int, entonces imprime "2-". Luego con `Run.execute(100L)`, ese es un long, y como no hay método con long, no lo puede convertir a int porque sería reducir, entonces lo convierte a Long (autoboxing) y termina usando el método de Object, por eso imprime "4-".

Ahora, si quitas el método de int, entonces para el 100 Java busca la siguiente mejor opción y prefiere autoboxing sobre varargs, así que usa Integer y imprime "3-", mientras que el long sigue cayendo en Object, o sea "4-".

entonces queda: 2- 4- normalmente, y si quitas el de int es 3- 4-

21. _B y D

La respuesta es B y D. O sea, aquí lo importante es entender qué significa realmente "sobrecargar" (overload): tienes que cambiar la lista de parámetros, no importa el nombre de las variables, el return o si es public o private.

Entonces, la A no cuenta porque aunque cambie nombres (a, b), los tipos son exactamente iguales (int, int...), o sea es la misma firma. La B sí cuenta porque cambia completamente el parámetro (char), entonces es otro método distinto. La C y la E no cuentan porque ni siquiera se llaman igual (moooo), entonces no es overload. La D sí cuenta porque aunque solo tenga int..., ya es una lista de parámetros diferente, y además el private no importa. Y F y G están mal porque el varargs siempre tiene que ir al final y solo puede haber uno. entonces básicamente: para overload lo que manda es cambiar los parámetros, y aquí los válidos son B y D